图片理解-绘画分析

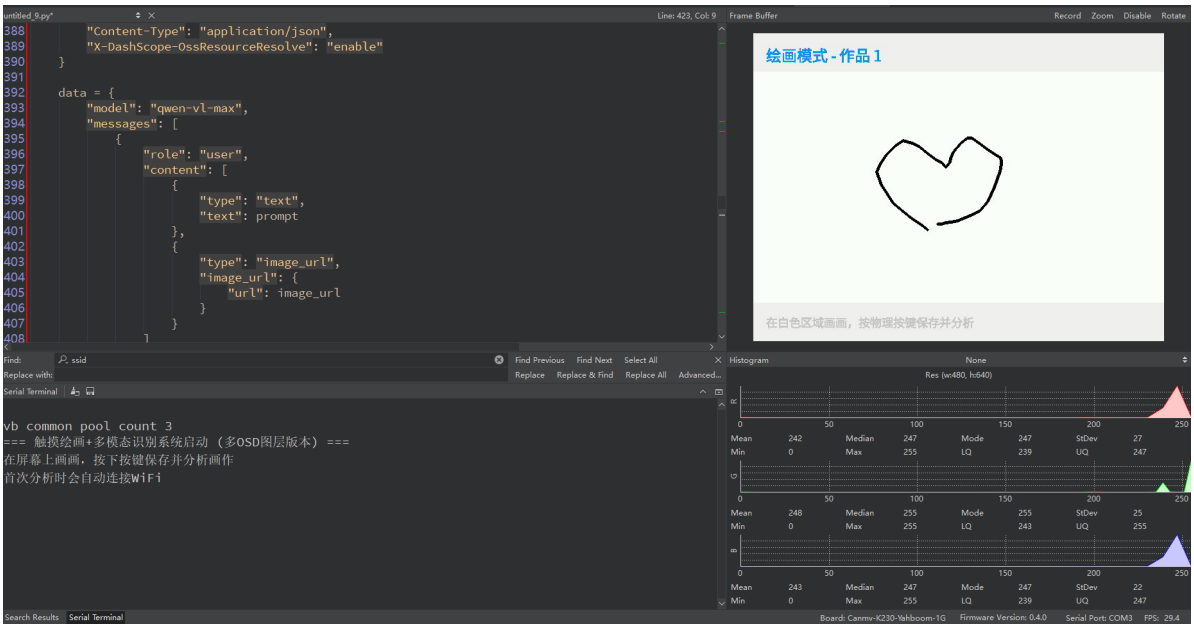
图片理解-绘画分析

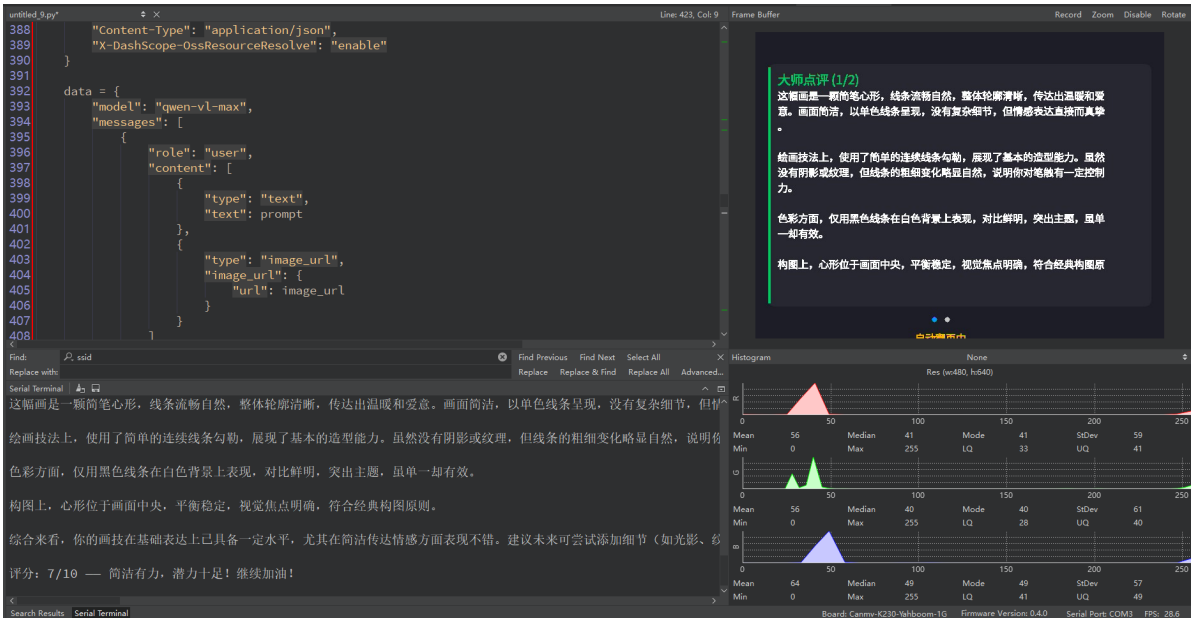
- 例程效果
- 代码讲解
  1. 导入必要的模块
  2. 定义全局变量和常量
  3. 辅助函数
  4. 主程序逻辑
  5. 异常处理与资源释放

例程效果

本节例程源码文件名为 `drawing_ai_analysis.py`。

连接IDE，在IDE中运行代码，程序使用触摸屏支持自由绘画，按下按键时保存画作，自动连接WiFi（首次使用），上传并使用AI进行多模态分析（绘画描述和评分）。





代码讲解

1. 导入必要的模块

```
import uos
import time
import ssl
from media.display import *
from media.media import *
import ybUtils.YbKey as YbKey
from machine import TOUCH
import libs.upload_image as upload_image
import YbRequests as requests
```

- `uos`：用于操作系统相关操作，如退出点。
- `time`：用于时间控制和延时。
- `ssl`：用于安全套接字层，支持HTTPS。
- `media.display`，`media.media`：用于屏幕显示和媒体管理。
- `ybUtils.YbKey`：用于按键检测。
- `machine.TOUCH`：用于触摸屏输入。
- `libs.upload_image`：自定义模块，用于图像上传。
- `YbRequests`：用于发送HTTP请求。

2. 定义全局变量和常量

```
save_path = "/sdcard/drawings/"
prefix = time.ticks_us() % 10000
drawing_count = 1
API_KEY = ""
WIFI_SSID = ""
WIFI_KEY = ""
network_connected = False
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
tp = TOUCH(0)
drawing_canvas = None
ui_overlay = None
UI_COLORS = { ... }
```

- `save_path`，`prefix`，`drawing_count`：用于保存绘图文件。
- `API_KEY`，`WIFI_SSID`，`WIFI_KEY`：API和WiFi配置。
- `network_connected`：网络连接状态。
- `tp = TOUCH(0)`：初始化触摸屏。
- `drawing_canvas`，`ui_overlay`：用于多OSD层管理。
- `UI_COLORS`：字典，定义界面颜色方案。

3. 辅助函数

- `draw_rounded_rect()`：绘制圆角矩形。
- `draw_progress_bar()`：绘制进度条，支持文本。
- `draw_status_card()`：绘制状态卡片。
- `show_loading_animation()`：显示加载动画。

- `show_progress_with_animation()`：显示带动画的进度界面。
- `create_drawing_canvas()`：创建绘画图层。
- `create_ui_overlay()`：创建UI覆盖层。
- `draw_on_canvas()`：在画布上绘制线条或点。
- `enhanced_network_connection()`：增强的WiFi连接函数，支持进度显示。
- `enhanced_ai_analysis()`：增强的AI分析函数，支持重试和错误处理。
- `analyze_image_with_dashscope()`：使用DashScope API分析图像。
- `enhanced_display_analysis_result()`：显示分析结果，支持分页。
- `smart_text_processing()`：智能处理文本。
- `split_text_to_pages()`：将文本分页。
- `ensure_dir()`：确保目录存在。

4. 主程序逻辑

```
if __name__ == "__main__":
    try:
        key = YbKey.YbKey()
        Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)
        MediaManager.init()
        drawing_canvas = create_drawing_canvas()
        ui_overlay = create_ui_overlay()
        last_x = None
        last_y = None
        last_key_status = False

        print("=== 触摸绘画+多模态识别系统启动 (多OSD图层版本) ===")

        while True:
            uos.exitpoint()
            point = tp.read(1)
            if len(point):
                pt = point[0]
                if pt.event == 0 or pt.event == TOUCH.EVENT_DOWN or pt.event ==
TOUCH.EVENT_MOVE:
                    draw_on_canvas(drawing_canvas, pt.x, pt.y, last_x, last_y)
                    last_x = pt.x
                    last_y = pt.y
                else:
                    last_x = None
                    last_y = None

            Display.show_image(drawing_canvas, 0, 0, Display.LAYER_OSD1)
            Display.show_image(ui_overlay, 0, 0, Display.LAYER_OSD2)

            if key.is_pressed() == 1:
                if not last_key_status:
                    last_key_status = True
                    ensure_dir(save_path + str(prefix) + "/")
                    drawing_path = save_path + str(prefix) + "/" +
f"drawing_{drawing_count}.jpg"
                    drawing_canvas.save(drawing_path)
                    if not network_connected:
                        ip = enhanced_network_connection(WIFI_SSID, WIFI_KEY,
drawing_canvas)
```

```
                        if ip:
                            network_connected = True
                    if network_connected:
                        result = enhanced_ai_analysis(API_KEY, drawing_path, "请分析一下这幅画的内容，包括画面描述、绘画技法、色彩运用、构图特点等，并给我的画技打个分（1-10分），同时提供一些改进建议。请用简洁友好的语言回复，控制在300字以内。", drawing_canvas)
                        enhanced_display_analysis_result(drawing_canvas, result['choices'][0]['message']['content'], 10000)
                    drawing_count += 1
                    drawing_canvas = create_drawing_canvas()
                    ui_overlay = create_ui_overlay()
            else:
                last_key_status = False

            time.sleep(0.05)
    except KeyboardInterrupt as e:
        print(f"用户停止程序")
    except BaseException as e:
        print(f"发生异常: '{e}'")
    finally:
        Display.deinit()
        uos.exitpoint(uos.EXITPOINT_ENABLE_SLEEP)
        time.sleep_ms(100)
        MediaManager.deinit()
```

- 初始化按键、显示和媒体。
- 循环处理触摸事件，绘制在图层1。
- 显示UI在图层2。
- 按键触发保存绘图、分析和结果显示。
- 异常处理：清理资源。

5. 异常处理与资源释放

- 在主程序中使用 `try-except-finally` 块，确保显示和媒体资源被释放。